

5주차 진도보고서

AI 기반 유사 문서 검출 시스템





INDEX

목차

목차1

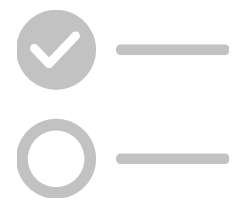
피드백

목차2

완료한 일

목차3

앞으로 할 일

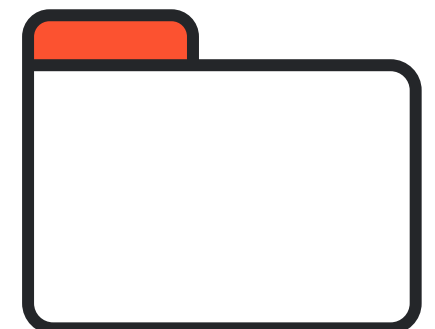




01

피드백

교수님 피드백 반영
팀장님 피드백 및 결론



9월 25일 피드백

교수님 피드백

- "기밀보고서를 직접 30개 정도 만들었다고 하는데, 굳이 만들 필요 없이 시중에 있는 투자 보고서를 이용하면 되지 않았냐"는 지적이 있었음.

요지

- 굳이 시간을 들여 보고서를 새로 제작할 필요가 있었는가에 대한 문제 제기
- 이미 시중에 다양한 투자 보고서 사례가 존재하므로, 그것을 활용하는 것이 더 효율적이라 판단

결론

- 시중 자료는 대부분 기밀성이 전혀 없고, PDF 형식에 한정되어 있어 다양한 파일 확장자를 다루기 어려움
- 따라서 실제 연구 및 실습 목적에 맞추어, 안전하면서도 다양한 형태의 기밀 보고서 데이터셋을 확보하기 위해 직접 제작할 수밖에 없었음

9월 18일 피드백

교수님 피드백 정리

주제가 "투자 보고서 기반 기밀 문서 유사도 탐지 시스템" 인데 일단은 "AI 기반 유사 문서 검출 시스템"로 하고 세부 사항으로 금융쪽 투자 보고서와 같이 해야 개발할 때 어디에 초점을 맞출 지 명확해질수 있다, 범용성을 넓혀라

요지

- 대주제는 범용적으로: AI 기반 유사 문서 검출 시스템
- 세부 적용 도메인은 예시로: 금융(투자 보고서)
- 개발 초점이 명확해지도록 범용성 확보 + 도메인 구체화 병행

결론

- 문서/발표 상의 프로젝트 표기는 "**AI 기반 유사 문서 검출 시스템**"으로 통일
- 세부 사례와 데이터셋·데모는 "**금융(투자 보고서)**"로 설명



소만사 팀장님께 피드백을 받기 위해 연락

확인후 회신드리겠습니다.

자세한 내용을 확인하기 전에 우선 의견을 답변드리겠습니다.

학생분이 설정하신 투자보고서는 제가 생각하지 못했던 분야의 데이터이며 그러한 데이터로 디파인 하신것은 매우 가치있는 판단으로 생각합니다.

회사마다 기밀문서는 모두 다를 수 있습니다. 디자인 회사, 쇼핑몰 회사, IT기업등 분야별로 다양할 수 있을 수 있습니다.

투자보고서라고 하신다면 스타트업이나 펀딩을 하는 회사 입장에서 고려하신거 같다는 느낌이 드네요. 아! 나중에 여쭙는게 좋겠다는 생각이며 우선은 매우 창의적인 설정이라고 생각합니다.

특정 도메인에 특화된 솔루션이 더욱 가치를 발휘할 수 있을 수 있으니 추가적인 논리와 설득력을 부여하시면 좋을 것 같습니다.

공유해주신 노선은 찬찬히 살펴보고 추가 피드백 드리도록 하겠습니다.

감사합니다.

안녕하세요. 소만사 김정환입니다.

금일 github 코드를 클론하여 실행해 보려 했지만 아쉽게도 template.hwpix 파일이 없어 실행을 해보진 못했습니다.

프로젝트 관련 조언드릴 내용은 다음과 같습니다.

pip install 로 여러 패키지를 설치 하고 실험하는 과정에서 시스템이 꼬일 수 있으니 uv 사용을 권장드립니다.

일단 **pip install uv** 명령어로 uv 를 설치하신 후 첨부해 드린 파일을 프로젝트에 복사해 넣으시기 바랍니다.

이후 다음과 같이 실행하시면 됩니다.

```
uv run fake_report_gemini.py --template
./template.hwpix --out ./output --count 5
```

기존 python 명령어 대신 uv run 으로만 하시면 됩니다.

```
python generate_reports_fake_kr_dynamic.py --
template ./template.hwpix --out ./out --count 5

=> uv run generate_reports_fake_kr_dynamic.py --
template ./template.hwpix --out ./out --count 5
```

신규 패키지 설치시에는 **pip install xxx** 대신 **uv add xxx** 로 하시면됩니다.

이와 같이 할 경우 프로젝트 폴더에 .venv 폴더가 생성되며 프로그램 실행 관련 라이브러리 들이 해당 폴더에서만 다운로드 및 실행되어 다른 라이브러리들과의 충돌이슈가 발생하지 않습니다.

github 에 commit 할 때는 .venv 디렉토리는 .gitignore 에 추가하여 저장소에 올라가지 않도록 하시면됩니다.

그럼 또 다른 문의 사항 있으시면 언제든지 문의 주시기 바랍니다.

감사합니다.



안녕하세요. 소만사 김성환입니다.

노선에 잘 정리하고 계신것 같아 특별한 코멘트는 없습니다.

전체적인 진행 과정에 대해서는 교수님께서 잘 지도해 주시고 계신것 같습니다.

회의록의 내용에서 교수님께서 범용성을 말씀해 주신것에 대해서는 일면 공감하는 부분은 있습니다.

어차피 문서의 유사도 판별은 도메인 특성과 상관없이 문서의 문맥과 키워드 일치도 등으로 유사도를 판별하면 되기 때문일 것입니다.

이러한 관점에서 범용성을 말씀하신듯 합니다.

그럼에도 불구하고 한동균 학생이 특정 도메인(금융 투자 보고서)을 설정해서 진행하는 점에 대해서는 긍정적인 의견입니다.

왜냐하면 회사마다 기밀 문서는 전부 다를 수 있기 때문입니다. 디자인 회사, 건설 도면 설계 회사등인 경우에는 문맥이나 키워드 보다는 이미지 유사도를 판별해야 할 수 있기 때문입니다.

개발 회사라면 소스코드 위주의 특화된 기능이 필요할 수 있기 때문에 너무 범용적이며 일반적인 구현 결과물 보다는 Domain Specific 한 결과로 설득력을 갖추신다면 좋을 것 같습니다.

정리하자면 일반적인 유사 문서 판단도 가능하지만(유사 문서 판단의 일반적인 구현), 금융 관련 회사에서는 특별히 더 판단력이 상승되는 특화 포인트를 강조하는 방향으로 프로젝트를 진행하시면 좋을 것 같습니다.

중간 중간 문의 사항이 있으시면 언제든지 추가 문의해주세요.

감사합니다.

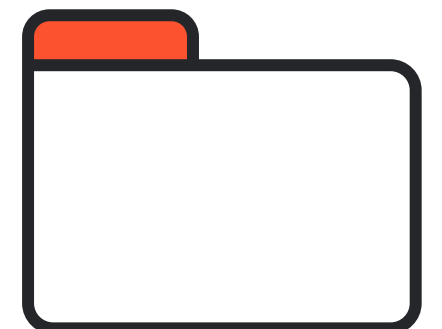
**결론적으로 금융 관련 회사에만
특화된 기능을 더 강조하는 방향으로
진행 예정!!**



02

완료한 일

수정된 WBS
hwp/hwpX 파서 구현
pdf 파서 구현
기밀 문서에 이미지 추가
paser 코드에 ocr 추가
임베딩 코드
임베딩 결과 예시



수정된 WBS

[illegible]

한글 파서 구현

```
# -*- coding: utf-8 -*-
# hwp_flat_lines_json_v2.py
# - .hwp(HWP5, OLE) / .hwp(X(ZIP+XML)) 자동 감지
# - 문서와 유사하게 "세로로 한 줄씩" JSON 배열에 담아서 출력
# - HWPX: p/para/paragraph/br/tbl/tr/tc/title 등에서 줄바꿈
# - HWP : 문단 텍스트 레코드(66)마다 줄바꿈
# - 옵션: --hard-wrap N (긴 줄 강제 줄나눔)

#py -3.13 -X utf8 hwp_flat_lines_json_v2.py "C:\Users\LG\조나우배터리 투자 보고서.hwpX" --oi

from __future__ import annotations
import sys, struct, zlib, json, re
from pathlib import Path
import zipfile, xml.etree.ElementTree as ET

# 콘솔 UTF-8
if hasattr(sys.stdout, "reconfigure"):
    try: sys.stdout.reconfigure(encoding="utf-8")
    except Exception: pass

# hwp(OLE) 선택 의존성
try:
    import olefile
    _HAS_OLE = True
except Exception:
    _HAS_OLE = False

OLE_SIG = b"\xD0\xCF\x11\xE0"
ZIP_SIG = b"PK\x03\x04"
HWPTAG_PARA_TEXT = 66

def detect_container(p: Path) -> str:
    if not p.is_file():
        raise FileNotFoundError(f"파일을 찾을 수 없습니다: {p}")
```

hwp/hwpX 파서 구현



한글(HWP/HWPX) 문서를 파싱하여
텍스트 데이터를 추출

HWP/HWPX/DOCX/PDF/이미지 등 다양한 입력을 AI
가 읽을 수 있는 일관된 스키마로 표준화

.pdf Paser

```
# -*- coding: utf-8 -*-
import pdfplumber
import json
import re
import unicodedata

# PDF에 자주 쓰이는 원형 글머리(일반 + 심볼/PUA) 후보들
# (☐, ☐ 등은 실제로 U+F06C, U+F06E 같은 PUA 문자)
BULLET_CHARS = ''.join([
    '•', '◦', '◐', '◑', '◒', '◓', '◔',
    '☐', '☑', '☒', '☓', '☔', '☕', '☖', '☗', '☘'
])

# 줄 맨 앞의 글머리 + 공백/탭/점 등을 제거
BULLET_START_RE = re.compile(rf'^[\s]*[{re.escape(BULLET_CHARS)}]+[\s·\t]*')

def clean_line(line: str) -> str:
    # 글꼴/폭 normalize (심볼이 일반 글리프로 매핑되는 경우 대비)
    s = unicodedata.normalize('NFKC', line)
    # 줄 맨 앞의 글머리만 제거
    s = BULLET_START_RE.sub('', s)
    return s.strip()

def pdf_to_text(pdf_path, out_json="parsed_pdf.json"):
    all_texts = []
    with pdfplumber.open(pdf_path) as pdf:
        for i, page in enumerate(pdf.pages, 1):
            text = page.extract_text()
            if not text:
                continue
            # 줄 단위 → 글머리 제거 → 빈 줄 제외
            lines = []
            for raw in text.split("\n"):
                cleaned = clean_line(raw)
```

pdf 파서 구현



PDF 문서를 파싱하여 텍스트 데이터를 추출

PDF 문서를 파싱하여 본문 텍스트를 추출·정제하고
JSON 형식으로 변환

불필요한 특수문자와 글머리 기호를 제거해 AI가 학습·
검색하기 좋은 데이터셋을 생성

매출	3957억원	4985억원	5867억원	5539억원
시장 점유율	18.8%	1.0%	13.4%	조나우배터리
주요 고객사	완성차	ESS	완성차	ESS
핵심 기술	셀/모듈/팩	셀/모듈/팩	셀/모듈/팩	셀/모듈/팩
주요 리스크	고객 집중	원자재 가격 변동	고객 집중	원가/환율/규제

○ 추가 경쟁사 정보 : 신규 진입자 · 대체재 동향 주기적 점검

4 규제 및 정책 리스크

- 현행 규제 환경 : 환경 · 안전 규제 · 인증 요건 존재
- 향후 정책 변화 : 지원정책/규제완화 · 강화 불확실성
- 리스크 요약 : 인증 · 승인 지연 시 매출 인식 지체 가능

V 재무 분석

1 과거 재무 성과 (3~5년)

○ 손익계산서 요약 (P/L)

구분	2023	2024	2025
매출액 (억원)	4322	4908	5539
영업이익 (억원)	468	636	836
당기순이익 (억원)	220	250	282
영업이익률 (%)	10.8	13.0	15.1

○ 매출액: 2023~2025 매출 4322→5539억 (CAGR 15.3%)

기밀 문서에 이미지 추가



OCR 처리를 위해서 30개의 문서에 이미지 추가

이미지에 들어갈 기밀 내용이 딱히 없다고 판단해, 기존에 있던 표를 스크린샷 해서 첨부

.docx Paser(OCR 포함)

```
import json
import re
import os
from docx import Document
from docx.text.paragraph import Paragraph
from docx.table import Table
from docx.xml.table import CT_Tbl
from docx.xml.text.paragraph import CT_P
import easyocr

# EasyOCR (컬러/손글씨/썸네일용)
easy_reader = easyocr.Reader(['ko', 'en'])

# 문장 분리 함수 (본문용)
def splitSentences(text):
    sentences = re.split(r"[\n]", text)
    return [s.strip() + "." for s in sentences if s.strip()]

# OCR 수행 (EasyOCR)
def parseImageOCR(imgPath, save_dir):
    imgPath = imgPath.replace("\\", "/")
    try:
        results = easy_reader.readtext(imgPath, detail=0)
        if results:
            print(f"{os.path.basename(imgPath)} → EasyOCR 인식 성공 ({len(results)}개 문장)")
            return results
    except Exception as e:
        print(f"EasyOCR 오류: {e}")

    print(f"❌ {os.path.basename(imgPath)} → 인식 실패")
    return []
```

paser 코드에 OCR 포함



이미지를 텍스트화 하기 위해 OCR 포함

EasyOCR을 활용해서 이미지에 있는 텍스트를 json 형식으로 추출

임베딩 코드

```
import json
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModel
from pathlib import Path

# 1. BGE-M3 모델 로드
MODEL_NAME = "BAAI/bge-m3"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModel.from_pretrained(MODEL_NAME)

def get_embedding(text: str):
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)
    with torch.no_grad():
        embeddings = model(**inputs).last_hidden_state[:, 0, :] # CLS 토큰
    return embeddings[0].cpu().numpy().astype("float32")

# 2. JSON 로드
input_path = Path("parsed_docx.json")
with open(input_path, "r", encoding="utf-8") as f:
    data = json.load(f)

#list / dict 모두 대응
if isinstance(data, list):
    documents = data
elif isinstance(data, dict):
    documents = data.get("documents", [])
else:
    raise ValueError("지원하지 않는 JSON 구조입니다.")
```

임베딩 코드



임베딩 코드

BGE-M3 모델을 사용해 문서 텍스트를 고차원 벡터로 변환

추출된 임베딩을 기반으로 FAISS 인덱스에 저장·검색 가능

임베딩 결과 예시

```
"text": "그린퓨처 투자 분석 보고서.",  
"vector": [  
  -2.293227195739746,  
  0.7665217518806458,  
  -0.7875639796257019,  
  0.3285672962665558,  
  -1.2287414073944092,  
  -0.3491098880767822,  
  -0.07513583451509476,  
  0.06582771241664886,
```




03

앞으로 할 일

벡터 스코어 구축
유사 검색 AI
간이 UI



벡터 스코어 구축



FAISS(HNSW/IVF) 인덱스, upsert/삭제/리빌드

- 대규모 데이터 빠른 검색을 위한 인덱스 구조 설계

Upsert/삭제/리빌드로 유연한 데이터 관리

실시간 검색 성능 최적화 기반 마련



유사 검색 AI



조각 Top-k(예:5) 검색, 코사인 유사도 반환

- 질의 문장과 가장 유사한 Top-k 결과 반환

코사인 유사도로 직관적 유사도 확인 가능

다양한 활용(추천, 검색, 분류)에 적용 가능



간이 UI



파일 업로드/Top-k/근거 하이라이트 간이 화면

- 파일 업로드 및 Top-k 결과 간단 확인

검색 결과 근거 하이라이트 제공

직관적이고 빠른 프로토타입 검증 가능



QnA

